

Hands-On Lab 3 : Understanding Git Branching

Learning Objective

- Understand what a branch is in Git.
 - Learn why branches are used.
 - Create and switch branches.
 - Merge branches into master.
-

Learning Outcome

After completing this lab, you will be able to:

- Create multiple branches.
 - Switch between branches.
 - Understand branch isolation.
 - Merge branches successfully.
-

Concept Explanation

What is a Branch?

A branch is an independent line of development.

It allows multiple developers to:

- Work on different features.
- Fix bugs independently.
- Experiment without affecting main code.

By default, Git creates a branch called `master` (or `main`).

Why Do We Use Branches?

Feature development , Bug fixing , Parallel development , Safe experimentation

Example:

- `master` → production-ready code

- branch 1 → feature A
 - branch 2 → feature B
-

Lab Steps

Step 1 : Create Repository

```
mkdir branch_lab  
cd test  
git init
```

Create a file and commit:

```
vi loginfile  
git add loginfile  
git commit -m "first commit"
```

Step 2 : Create Branches

```
git branch branch1  
git branch branch2  
git branch
```

Step 3 : Switch to branch 1

```
git switch branch1
```

Create a file:

```
vi branch1-file  
git add branch1-file  
git commit -m "branch1 commit"
```

Step 4 : Switch to branch 2

```
git switch branch2
```

Notice: branch 1 -file is not visible here.

Create another file:

```
vi branch2_file  
git add branch2_file  
git commit -m "branch2 commit"
```

Step 5 : Merge branch 1 into master

```
git switch master  
git merge branch1
```

Step 6 : Merge branch 2

```
git merge branch2
```

Important Understanding

Concept	Meaning
Branch	Separate line of development
git branch	Create branch
git switch	Move between branches
git merge	Combine branches